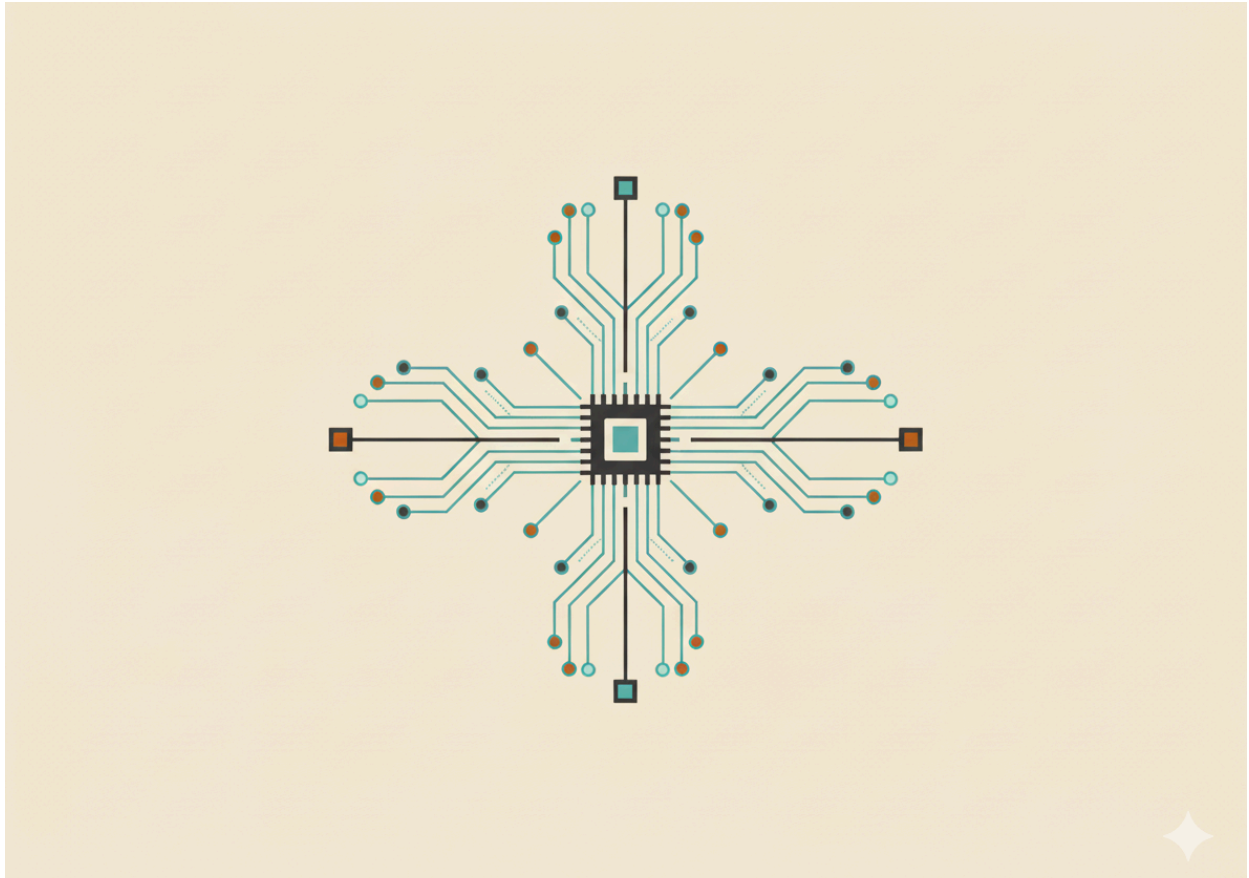


Trabajo Práctico 2: Memoria



Ariel Eduardo Schenone Duarte

LU: 431/22
2do Cuatrimestre 2025
Sistemas Operativos

PARTE 1

Ejercicio 1

Inciso a

Voy a presentar un ejemplo de lo que debería devolver cada operación de Alloc y/o Free.

Para cada acción de Alloc, se devuelve una dirección de memoria en caso de éxito o el valor -1 en caso de que la asignación no sea exitosa.

3ra Operación:

- `ptr[1] = Alloc(5)` returned 1003

En cambio para la operación de liberar memoria, se retorna el valor 0.

8va Operación:

- `Free(ptr[3])` returned 0

Para consultar con detalle el resultado de cada operación de la ejecución consultar el [Apéndice A](#).

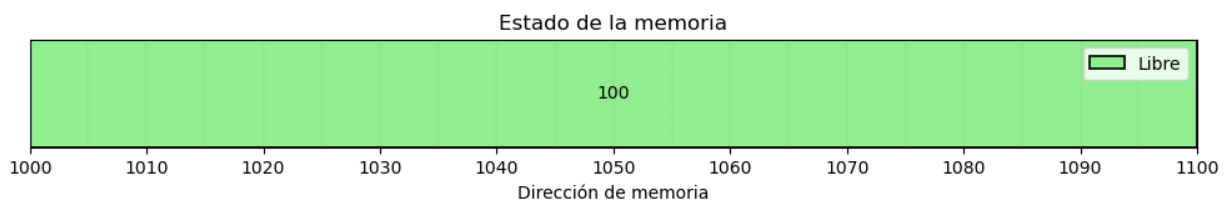
Inciso b

Durante las 10 operaciones(6 allocs y 4 free) la estructura de la free list se ve modificando conforme se va asignando o liberando memoria.

Para evitar incluir datos repetitivos o extensos, el detalle completo de cada operación y como está en ese momento la estructura de la free list consultar el [Apéndice B](#).

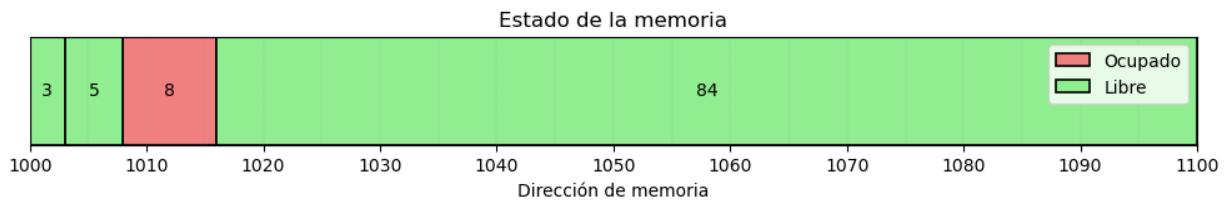
A continuación se presentan mediante gráficos como se ve la free list en tres momentos representativos, al principio, por la mitad del proceso y al finalizar.

Inicio



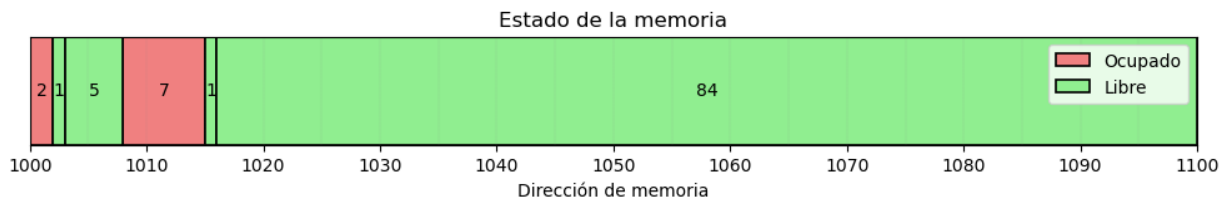
Como se puede observar al comienzo de la ejecución toda la memoria está disponible y unificada sin fragmentación.

Mitad



Se puede observar ya en este punto como quedó fragmentada la memoria luego de algunas liberaciones de memoria que fue asignada previamente. Aún se mantiene un bloque lo suficientemente grande para poder devolver futuras asignaciones de memoria.

Final



Ahora sí se aprecia efectivamente la fragmentación de los bloques, llegando al punto de tener 2 bloques de tamaño 1, tamaños muy pequeños lo cual reduce la eficiencia y dificulta futuras asignaciones de tamaños medianamente grandes .

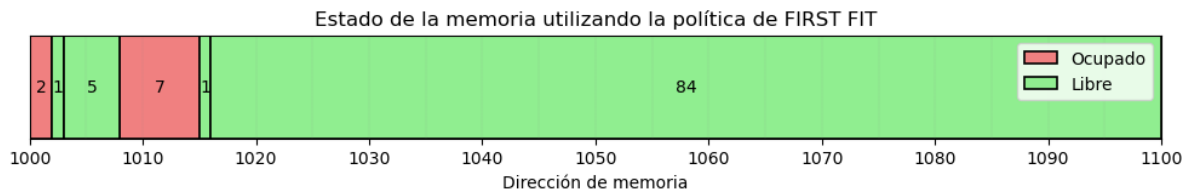
Inciso c

Al pasar cada operación de Alloc o Free, se puede apreciar como la free list se va agrandando en tamaño, ya que cada Alloc genera un bloque de cierto tamaño ocupado hasta que con el uso de Free ese bloque se libera y se agrega a la free list.

Un factor que favorece este comportamiento de la free list es que no estamos usando la opción de coalescing, el cual permite juntar o combinar los bloques libres continuos, cuyo resultado es una menor cantidad de bloques en la free list.

Ejercicio 2

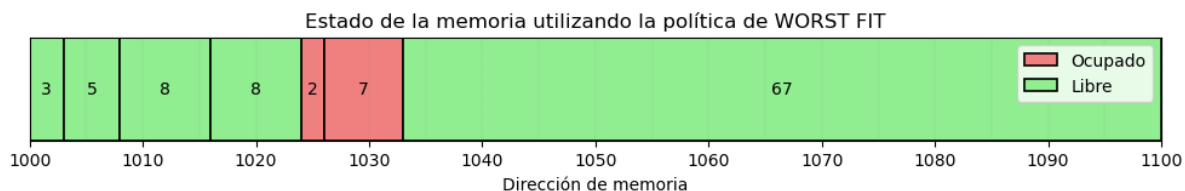
Análisis con la política de FIRST FIT



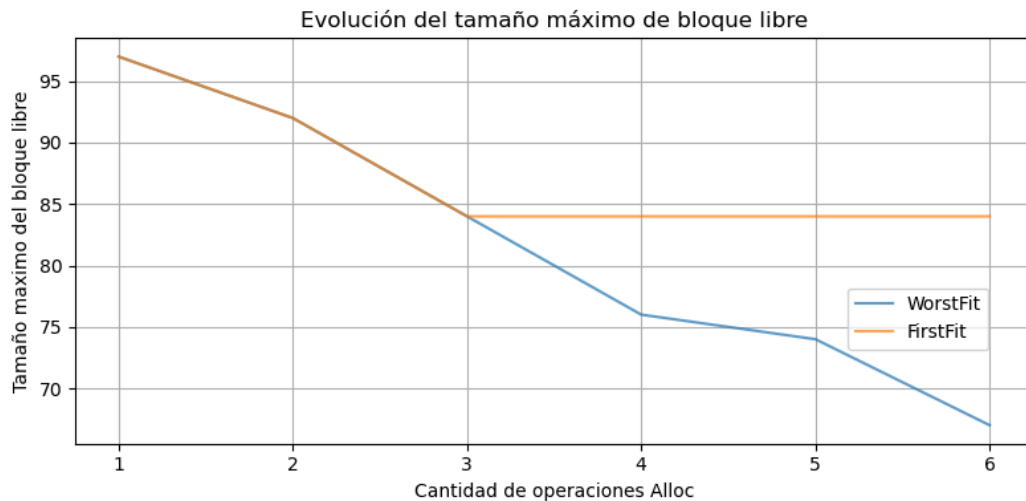
Podemos observar que las políticas de **BEST FIT** y **FIRST FIT** generan la misma estructura de la free list, debido a que para ambas políticas la mejor opción era la misma en cada paso. Aunque si hablamos en términos de velocidad **FIRST FIT** es más rápido ya que no recorre toda la estructura de la free list.

Análisis con la política de WORST FIT

En el caso donde se utilizó la política de **WORST FIT** si podemos observar una diferencia más grande, debido a que esta política cada vez que se pide memoria elige el bloque libre más grande, sin importar si hay uno más ajustado, fragmentando la memoria desde el extremo más grande.

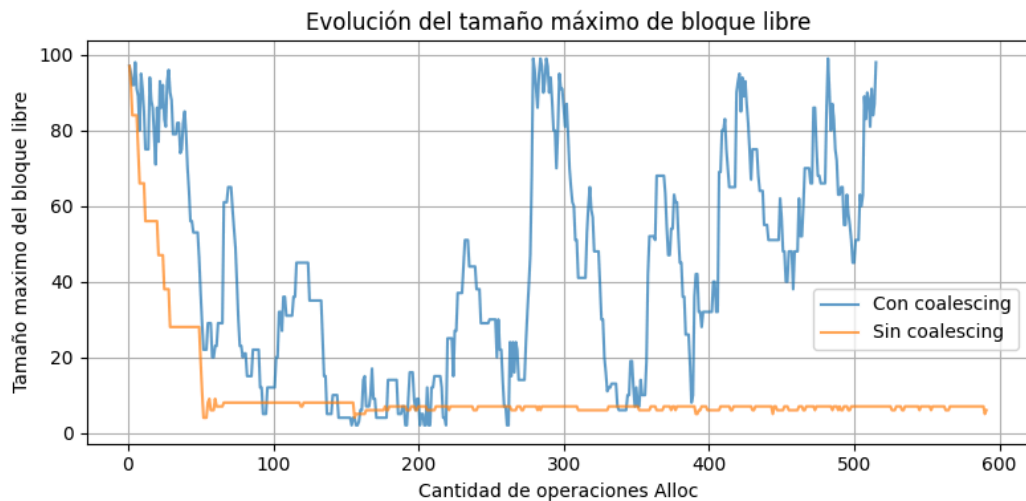


Pese a ser un caso de prueba pequeño con el siguiente gráfico podemos observar como la ejecución que utilizó la política de **WORST FIT** decrementa mucho más rápido el tamaño de su bloque libre más grande debido a que siempre **WORST FIT** va a elegir para asignar el bloque con mayor tamaño lo que implica que por cada asignación el bloque más grande decrementa su tamaño.



Ejercicio 3

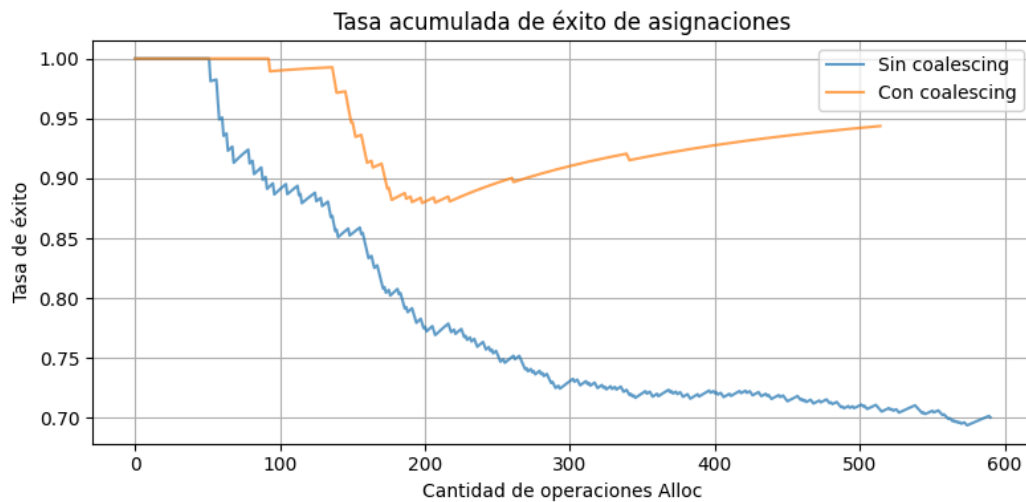
Al incrementar la cantidad de operaciones de asignación aleatoria y al no usar la opción de coalescing, que combina los bloques libres continuos, lo que se genera es que los bloques más grandes se irán fragmentando cada vez más en bloques más pequeños.



Un problema de tener muchas operaciones de asignación de memoria y tener cada vez bloques más pequeños, aumenta la probabilidad de no poder devolver una asignación correcta para la cantidad de espacio que te piden, por ejemplo al tener esa misma free list y te piden un bloque de tamaño 8, no puedes devolver ninguna dirección.

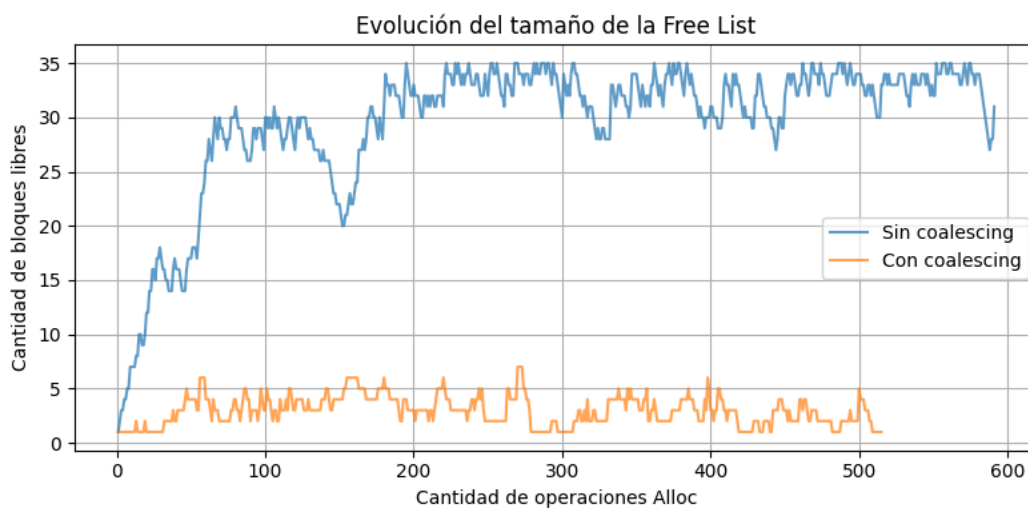
En cambio al utilizar la opción de coalescing vamos a mantener bloques más grandes,

gracias a que los bloques libres continuos se fusionan, permitiendo devolver direcciones de pedidos de memoria medianamente grandes.



Con este gráfico podemos observar como el success rate se mantiene alto cuando se utiliza la opción de coalescing, en cambio en el caso de no usar el coalescing se ve como el success rate baja alrededor de un 30%.

Otra diferencia importante es la variación del tamaño de la free list, donde en los casos donde se usa la opción de coalescing se mantiene una estructura pequeña, con pocos bloques y por otra parte al no usar coalescing y al no poder unificar los bloques libres continuos, la estructura de la free list aumenta en cantidad de bloques libres.



Para los gráficos de este ejercicio se ejecutó el comando: `-n 1000 -H 0 -p BEST -s 0`

PARTE 2

Ejercicio 1

Opciones ejecutadas: -s 0 -n 10

FIFO

Definición: Las páginas se reemplazan en el **orden en que fueron cargadas**. La más antigua es la primera en salir. Implementación simple mediante una cola circular.

Ventajas: Fácil de implementar.

Desventajas: No considera la frecuencia de uso; puede expulsar páginas muy activas.

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
8	[]	Miss	-	[8]
7	[8]	Miss	-	[8, 7]
4	[8, 7]	Miss	-	[8, 7, 4]
2	[8, 7, 4]	Miss	8	[7, 4, 2]
5	[7, 4, 2]	Miss	7	[4, 2, 5]
4	[4, 2, 5]	Hit	-	[4, 2, 5]
7	[4, 2, 5]	Miss	4	[2, 5, 7]
3	[2, 5, 7]	Miss	2	[5, 7, 3]
4	[5, 7, 3]	Miss	5	[7, 3, 4]
5	[7, 3, 4]	Miss	7	[3, 4, 5]

Total hits: 1 (solo el segundo acceso a la página 4)

Hit rate: 10%

Total misses: 9

Hit rate: 90%

LRU

Definición: Reemplaza la página que no se ha usado por más tiempo.

Ventajas: excelente rendimiento teórico; se aproxima al óptimo.

Desventajas: implementación costosa (necesita hardware o estructuras adicionales para registrar el orden de acceso).

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
8	[]	Miss	-	[8]
7	[8]	Miss	-	[8, 7]
4	[8, 7]	Miss	-	[8, 7, 4]
2	[8, 7, 4]	Miss	8	[7, 4, 2]
5	[7, 4, 2]	Miss	7	[4, 2, 5]
4	[4, 2, 5]	Hit	-	[2, 5, 4]
7	[2, 5, 4]	Miss	2	[5, 4, 7]
3	[5, 4, 7]	Miss	5	[4, 7, 3]
4	[4, 7, 3]	Hit	-	[7, 3, 4]
5	[7, 3, 4]	Miss	7	[3, 4, 5]

Total hits: 2

Hit rate: 20%

Total misses: 8

Hit rate: 80%

Mejora un poco en comparación a la ejecución que utilizó la política de FIFO logrando obtener un hit más.

OPT

Definición: Reemplaza la página que se va a usar más lejos en el futuro.

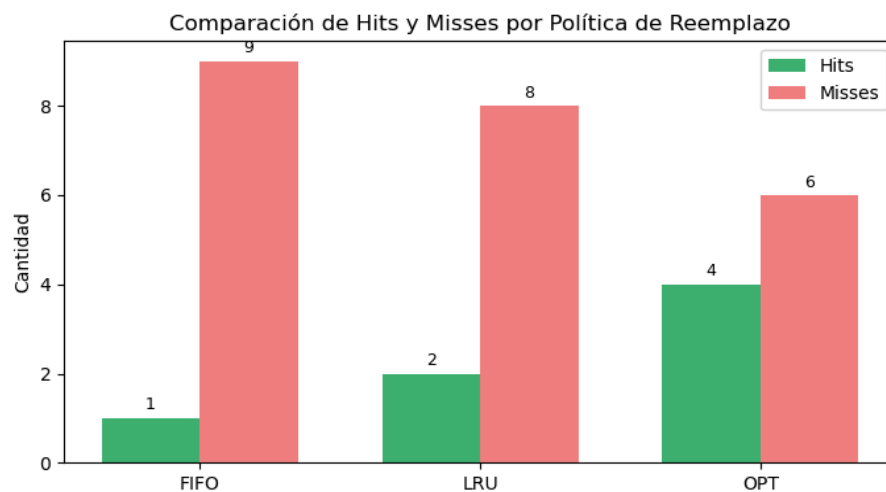
Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
8	[]	Miss	-	[8]
7	[8]	Miss	-	[8, 7]
4	[8, 7]	Miss	-	[8, 7, 4]
2	[8, 7, 4]	Miss	8	[7, 4, 2]
5	[7, 4, 2]	Miss	2	[7, 4, 5]
4	[7, 4, 5]	Hit	-	[7, 4, 5]
7	[7, 4, 5]	Hit	-	[7, 4, 5]
3	[7, 4, 5]	Miss	7	[3, 4, 5]
4	[3, 4, 5]	Hit	-	[3, 4, 5]
5	[3, 4, 5]	Hit	-	[3, 4, 5]

Total hits: 4

Hit rate: 40%

Total misses: 6

Hit rate: 60%



Opciones ejecutadas: -s 1 -n 10

En este caso las políticas de FIFO y LRU funcionaron de igual manera, produciendo las mismas estadísticas, ya que en cada paso la decisión tomada era la misma en las dos políticas .

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
1	[]	Miss	-	[1]
8	[1]	Miss	-	[1, 8]
7	[1, 8]	Miss	-	[1, 8, 7]
2	[1, 8, 7]	Miss	1	[8, 7, 2]
4	[8, 7, 2]	Miss	8	[7, 2, 4]
4	[7, 2, 4]	Hit	-	[7, 2, 4]
6	[7, 2, 4]	Miss	7	[2, 4, 6]
7	[2, 4, 6]	Miss	2	[4, 6, 7]
0	[4, 6, 7]	Miss	4	[6, 7, 0]
0	[6, 7, 0]	Hit	-	[6, 7, 0]

Total hits: 2

Hit rate: 20%

Total misses: 8

Hit rate: 80%

OPT

Definición: Reemplaza la página que se va a usar más lejos en el futuro.

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
1	[]	Miss	-	[1]
8	[1]	Miss	-	[1, 8]
7	[1, 8]	Miss	-	[1, 8, 7]
2	[1, 8, 7]	Miss	1	[2, 8, 7]
4	[2, 8, 7]	Miss	2	[4, 8, 7]
4	[4, 8, 7]	Hit	-	[4, 8, 7]
6	[4, 8, 7]	Miss	4	[6, 8, 7]
7	[6, 8, 7]	Hit	-	[6, 8, 7]
0	[6, 8, 7]	Miss	6	[0, 8, 7]
0	[0, 8, 7]	Hit	-	[0, 8, 7]

Total hits: 3

Hit rate: 30%

Total misses: 7

Hit rate: 70%

En cambio para el caso donde se decidió por el uso de la política OPT, el resultado tuvo una leve mejoría, aumentando en 1 la cantidad total de hits. Gracias a que por definición de esta política, se reemplazó una página que no volvía a ser referencia en lugar de sacar una página que volvía a pedirse un tiempo más tarde.

Opciones ejecutadas: -s 2 -n 10

En este caso el rendimiento de las 3 políticas fue el mismo, obteniendo los mismos resultados.

Por ejemplo, esta ejecución utilizando la política LRU.

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
9	[]	Miss	-	[9]
9	[9]	Hit	-	[9]
0	[9, 0]	Miss	-	[9, 0]
0	[9, 0]	Hit	-	[9, 0]
8	[9, 0]	Miss	-	[9, 0, 8]
7	[9, 0, 8]	Miss	9	[0, 8, 7]
6	[0, 8, 7]	Miss	0	[8, 7, 6]
3	[8, 7, 6]	Miss	8	[7, 6, 3]
6	[7, 6, 3]	Hit	-	[7, 6, 3]
6	[7, 6, 3]	Hit	-	[7, 6, 3]

Total hits: 4

Hit rate: 40%

Total misses: 6

Hit rate: 60%

Para más detalles sobre la ejecución de las otras políticas consultar en el [Apéndice C](#).

Ejercicio 2

Unas posibles secuencias para una ejecución que solo tiene 5 marcos de página podrían ser:

- [0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 0, 1, ...]
- [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, ...]

Una primera observación sería que al haber más referencias a páginas que marcos de páginas y a su vez entre cada llamado a la misma página hayan pasado por lo menos la cantidad de marcos de páginas disponibles, se generarían muchos misses.

El primer caso sería como una secuencia cíclica desplazada, que ocurre cuando cada página usada recientemente se vuelve a querer referenciar justo después de haber sido expulsada. Esta secuencia funciona como peor caso para las dos políticas de **FIFO** y **LRU**.

Para el segundo caso opte por una opción un poco más trivialmente mala, al tener el doble de referencias a páginas que marcos disponibles, cada intento de acceder generaría un miss constante en cada iteración.

Ahora para generar un rendimiento similar a si se utilizase la política de **OPT**, para el primer caso con agregar una página más no tendríamos problemas con los miss, pasarían a ser todos hit, debido a que la cantidad de marcos de páginas serían igual la cantidad de referencias de páginas.

Por otro lado, en el segundo caso para que las referencias a páginas pasen a ser hits hace falta que la page cache pueda contener todas las páginas distintas del ciclo, por tanto haría falta aumentar los marcos de 5 a 10 para eliminar los misses por repetición.

A su vez **OPT** siempre consigue menos misses que **FIFO/LRU** para el mismo número de marcos, porque usa información futura. Sin embargo, si nuestro objetivo es eliminar la mayor cantidad de misses, el número de marcos necesario es igual al tamaño de la cantidad de referencias a páginas distintas usadas.

Ejercicio 3

Se generó una traza con localidad temporal [4, 3, 4, 5, 4, 5, 8, 5, 8, 6, 8, 6], debido a que por intervalos cortos de tiempo se hace referencia a un conjunto de páginas que se utilizan con frecuencia.

Si contamos con 3 marcos de páginas podemos realizar el siguiente análisis.

Al utilizar la política de LRU, que conserva en memoria las páginas recientemente usadas, reduciendo significativamente la cantidad de misses en este caso.

Podemos observar que sobre las páginas 4, 5 y 8 se hace referencia con frecuencia en un periodo corto de tiempo, lo que significa que se mantiene en los marcos de página evitando así una significativa cantidad de misses.

Por lo tanto las métricas finales sobre este caso son bastante favorables, con una cantidad total de hits de 7 y una cantidad total de misses de 5, logrando así un hit rate del 58.33%.

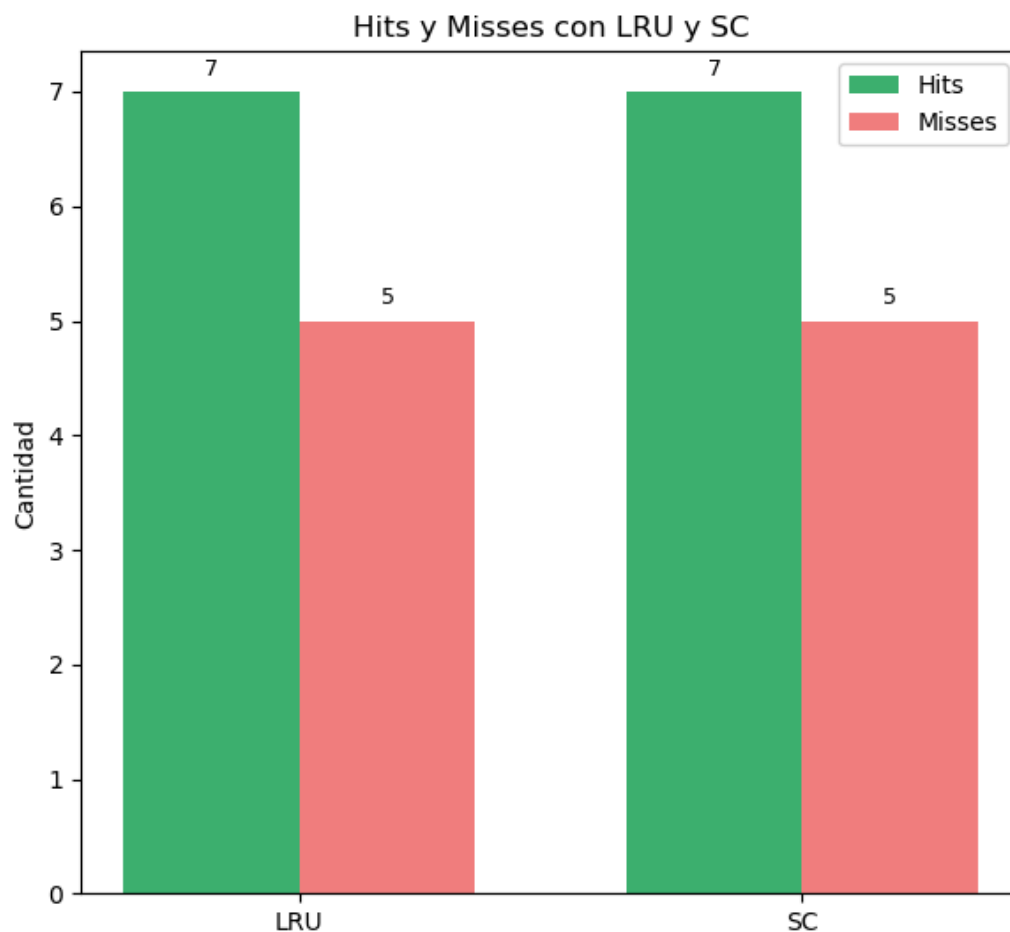
Por otro lado tenemos a la política de SECOND CHANCE que mantiene también una lista circular (tipo reloj) de páginas en memoria, cada una con un bit de referencia (0 o 1).

Este algoritmo como su nombre indica da una segunda oportunidad a las páginas usadas recientemente antes de reemplazarlas, imitando el comportamiento de LRU.

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
4	[]	Miss	-	[4]
3	[4]	Miss	-	[4, 3]
4	[4, 3]	Hit	-	[4, 3]
5	[4, 3]	Miss	-	[4, 3, 5]
4	[4, 3, 5]	Hit	-	[4, 3, 5]
5	[4, 3, 5]	Hit	-	[4, 3, 5]
8	[4, 3, 5]	Miss	3	[5, 4, 8]
5	[5, 4, 8]	Hit	-	[5, 4, 8]
8	[5, 4, 8]	Hit	-	[5, 4, 8]

6	[5, 4, 8]	Miss	4	[8, 5, 6]
8	[8, 5, 6]	Hit	-	[8, 5, 6]
6	[8, 5, 6]	Hit	-	[8, 5, 6]

Así con esta traza obtenemos que la cantidad de hits totales es 7 y la cantidad de misses totales es 5, con hit rate de 58.33%, lo cual nos dice que para este caso ambas políticas nos dan los mismos resultados, con un buen porcentaje de hit rate para ambos.



Para concluir, este análisis comparativo de la traza de referencia de páginas [4, 3, 4, 5, 4, 5, 8, 5, 8, 6, 8, 6] con capacidad para utilizar 3 marcos de página y empleando dos políticas distintas revela una convergencia notable en el rendimiento de los algoritmos LRU y SC.

Esto se produce gracias al gran impacto de la localidad temporal, cuya traza de referencia se caracteriza por un uso frecuente de un conjunto pequeño de páginas en cortos intervalos.

Apéndice A

1ra Operación:

- `ptr[0] = Alloc(3)` returned 1000

2da Operación:

- `Free(ptr[0])` returned 0

3ra Operación:

- `ptr[1] = Alloc(5)` returned 1003

4ta Operación:

- `Free(ptr[1])` returned 0

5ta Operación:

- `ptr[2] = Alloc(8)` returned 1008

6ta Operación:

- `Free(ptr[2])` returned 0

7ma Operación:

- `ptr[3] = Alloc(8)` returned 1008

8va Operación:

- `Free(ptr[3])` returned 0

9na Operación:

- `ptr[4] = Alloc(2)` returned 1000

10ma Operación:

- `ptr[5] = Alloc(7)` returned 1008

Apéndice B

1ra Operación:

- free list = [addr:1003 sz:97]

2da Operación:

- free list = [addr:1000 sz:3][addr:1003 sz:97]

3ra Operación:

- free list = [addr:1000 sz:3][addr:1008 sz:92]

4ta Operación:

- free list = [addr:1000 sz:3][addr:1003 sz:5][addr:1008 sz:92]

5ta Operación:

- free list = [addr:1000 sz:3][addr:1003 sz:5][addr:1016 sz:84]

6ta Operación:

- free list = [addr:1000 sz:3][addr:1003 sz:5][addr:1008 sz:8][addr:1016 sz:84]

7ma Operación:

- free list = [addr:1000 sz:3][addr:1003 sz:5][addr:1016 sz:84]

8va Operación:

- free list = [addr:1000 sz:3][addr:1003 sz:5][addr:1008 sz:8][addr:1016 sz:84]

9na Operación:

- free list = [addr:1002 sz:1][addr:1003 sz:5][addr:1008 sz:8][addr:1016 sz:84]

10ma Operación:

- free list = [addr:1002 sz:1][addr:1003 sz:5][addr:1015 sz:1][addr:1016 sz:84]

Apéndice C

FIFO

Definición: Las páginas se reemplazan en el **orden en que fueron cargadas**. La más antigua es la primera en salir. Implementación simple mediante una cola circular.

Ventajas: fácil de implementar.

Desventajas: no considera la frecuencia de uso; puede expulsar páginas muy activas.

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
9	[]	Miss	-	[9]
9	[9]	Hit	-	[9]
0	[9, 0]	Miss	-	[9, 0]
0	[9, 0]	Hit	-	[9, 0]
8	[9, 0]	Miss	-	[9, 0, 8]
7	[9, 0, 8]	Miss	9	[0, 8, 7]
6	[0, 8, 7]	Miss	0	[8, 7, 6]
3	[8, 7, 6]	Miss	8	[7, 6, 3]
6	[7, 6, 3]	Hit	-	[7, 6, 3]
6	[7, 6, 3]	Hit	-	[7, 6, 3]

Total hits: 4

Hit rate: 40%

Total misses: 6

Hit rate: 60%

LRU

Definición: Reemplaza la página que no se ha usado por más tiempo.

Ventajas: excelente rendimiento teórico; se aproxima al óptimo.

Desventajas: implementación costosa (necesita hardware o estructuras adicionales para registrar el orden de acceso).

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
9	[]	Miss	-	[9]
9	[9]	Hit	-	[9]
0	[9, 0]	Miss	-	[9, 0]
0	[9, 0]	Hit	-	[9, 0]
8	[9, 0]	Miss	-	[9, 0, 8]
7	[9, 0, 8]	Miss	9	[0, 8, 7]
6	[0, 8, 7]	Miss	0	[8, 7, 6]
3	[8, 7, 6]	Miss	8	[7, 6, 3]
6	[7, 6, 3]	Hit	-	[7, 3, 6]
6	[7, 6, 3]	Hit	-	[7, 3, 6]

Total hits: 4

Hit rate: 40%

Total misses: 6

Hit rate: 60%

OPT

Definición: Reemplaza la página que se va a usar más lejos en el futuro.

Acceso	Estado antes	Hit/Miss	Reemplaza a	Estado después
9	[]	Miss	-	[9]
9	[9]	Hit	-	[9]
0	[9, 0]	Miss	-	[9, 0]
0	[9, 0]	Hit	-	[9, 0]
8	[9, 0]	Miss	-	[9, 0, 8]
7	[9, 0, 8]	Miss	9	[0, 8, 7]
6	[0, 8, 7]	Miss	0	[8, 7, 6]
3	[8, 7, 6]	Miss	8	[7, 6, 3]
6	[7, 6, 3]	Hit	-	[7, 3, 6]
6	[7, 6, 3]	Hit	-	[7, 3, 6]

Total hits: 4

Hit rate: 40%

Total misses: 6

Hit rate: 60%